

Lecture 6

Virtual Memory

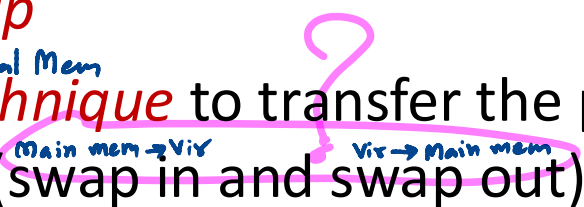
Previously..

- We are continue on Memory allocation not on Contiguous but on Noncontiguous memory allocation
- Main focus still on address translation (?)
- Noncontiguous allocation
 - Segmentation Scheme
 - Based and bound register-pair for each segment
 - Different size for each segments, all in the segment table
 - Paging Scheme
 - Same size for each pages, all in the page table
 - Page tables is big .. Invert page table and multi-level pages table
 - (Hybrid Paging and Segmentation Scheme)

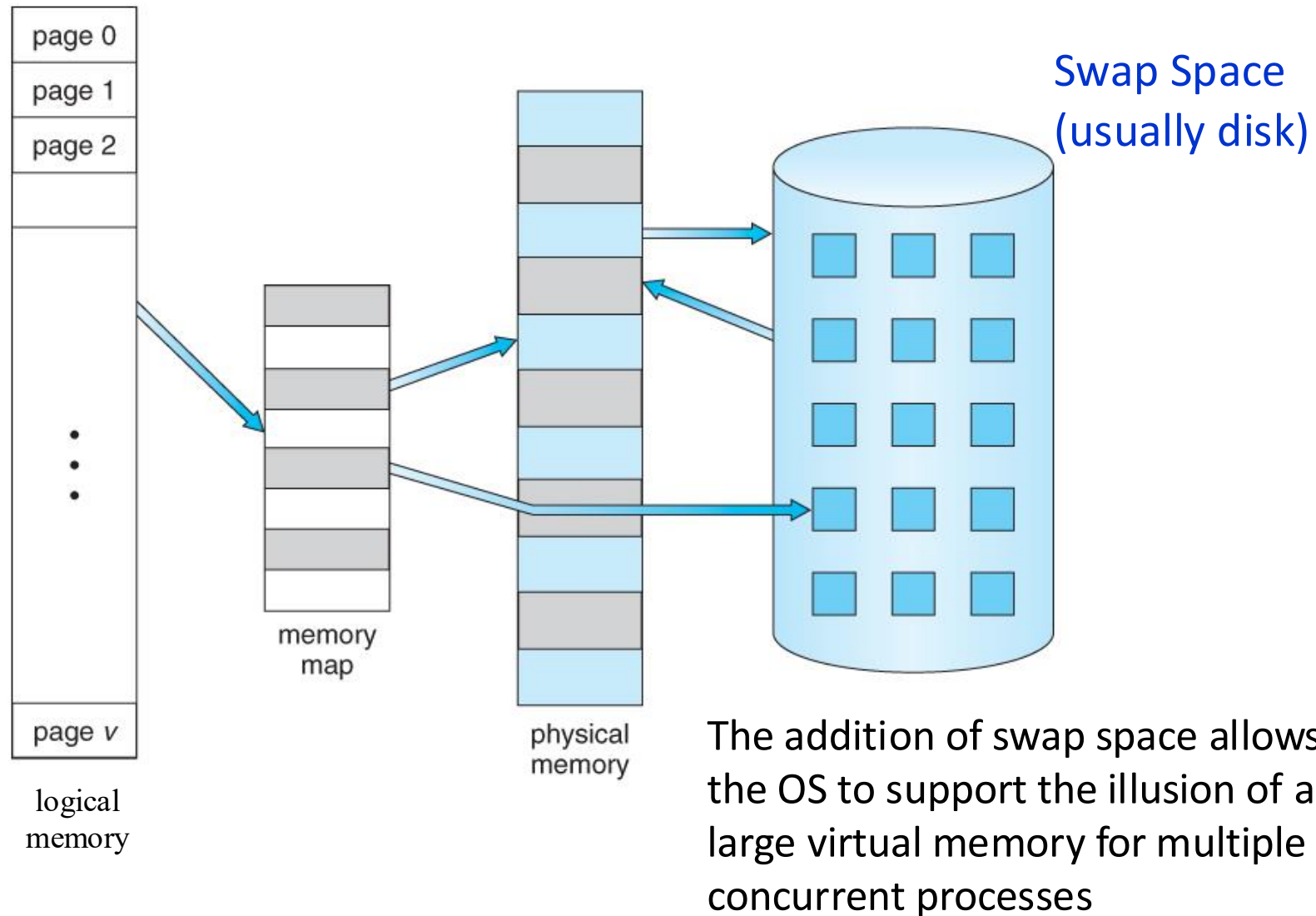
Introduction

- So far we have assumed that the combined address spaces of all running processes fit into physical memory
- Nowadays, applications are getting more complex and memory intensive and so have much larger address spaces
- The logical address space of a process may be too large to fit into physical memory
- The **Virtual Memory** concept is used to extend the physical memory space
- Job of OS: to organize the logical memory so that portions of address spaces that currently are not frequently used are stored outside the main memory

Virtual Memory Concept

- Use either *paging* or *segmentation* schemes (we will focus on the **paging scheme**; prior lecture)
- Use *backing storage (disk)* to keep pages that are not in use – need a **memory map**
- Use a **swapping technique** to transfer the pages between disk and main memory (swap in and swap out) $\neq$ Context Switch

Main Mem $\leftrightarrow$ Virtual Mem
Main mem $\rightarrow$ Vir Vir $\rightarrow$ Main mem
- Use a **page selection policy** to determine when a page is swapped in
Swap in/out
PCB on Running State $\leftrightarrow$ Ready State
- Use a **page replacement policy** to determine the best page to swap out
Swap out (victim page)

Page Mapping



Context switch

حالة البرنامج Process state

Main Mem ↔ Virtual Mem

Process Switching vs. Process Swapping

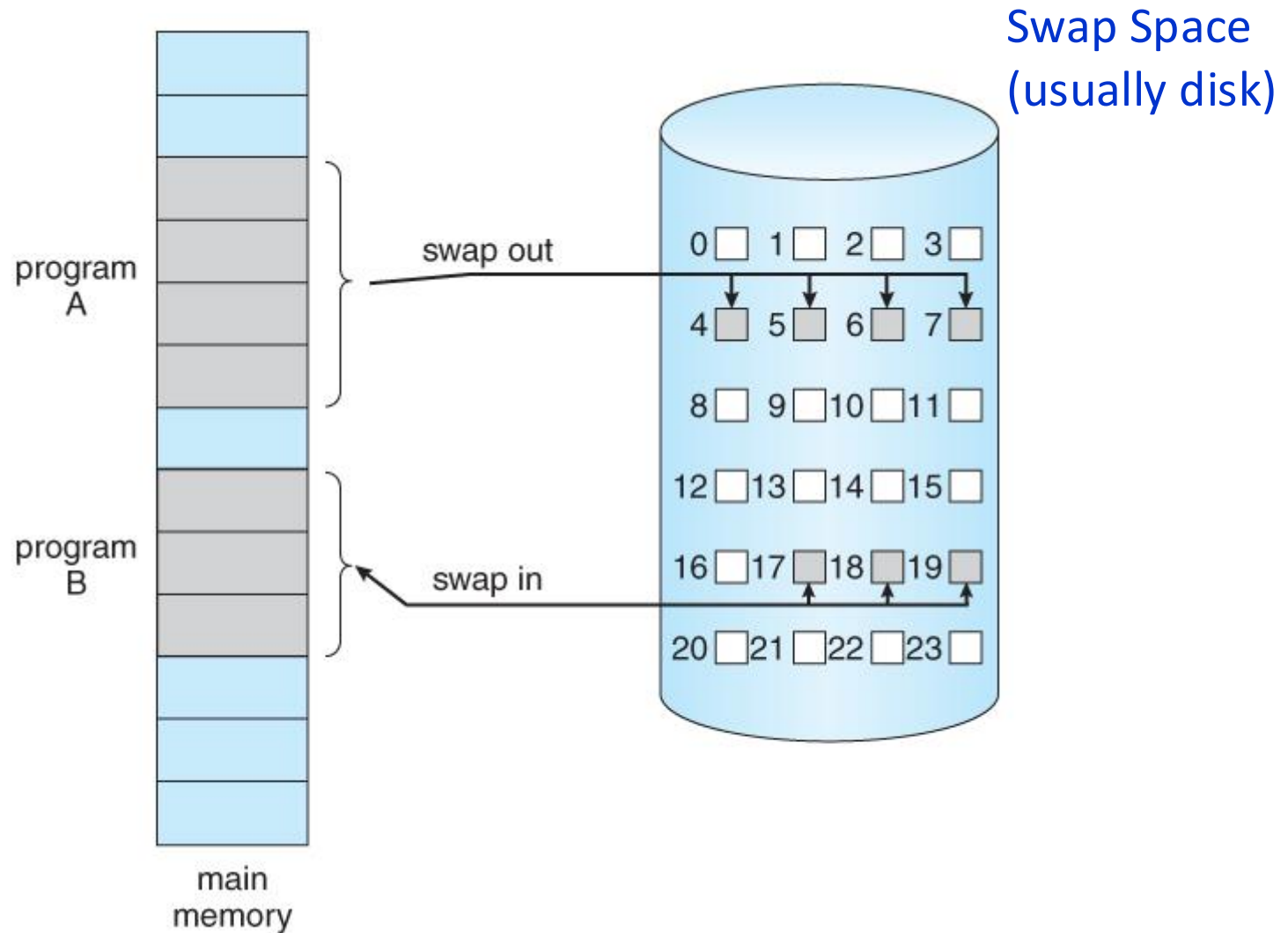
- Process Switching

- The kernel scheduler deschedules a process and the current context (including registers) is saved in memory
- The context of the next process to be executed is restored
- This action is also called *Context Switching*
- Review non-preemptive vs. preemptive schedulers

- Process Swapping

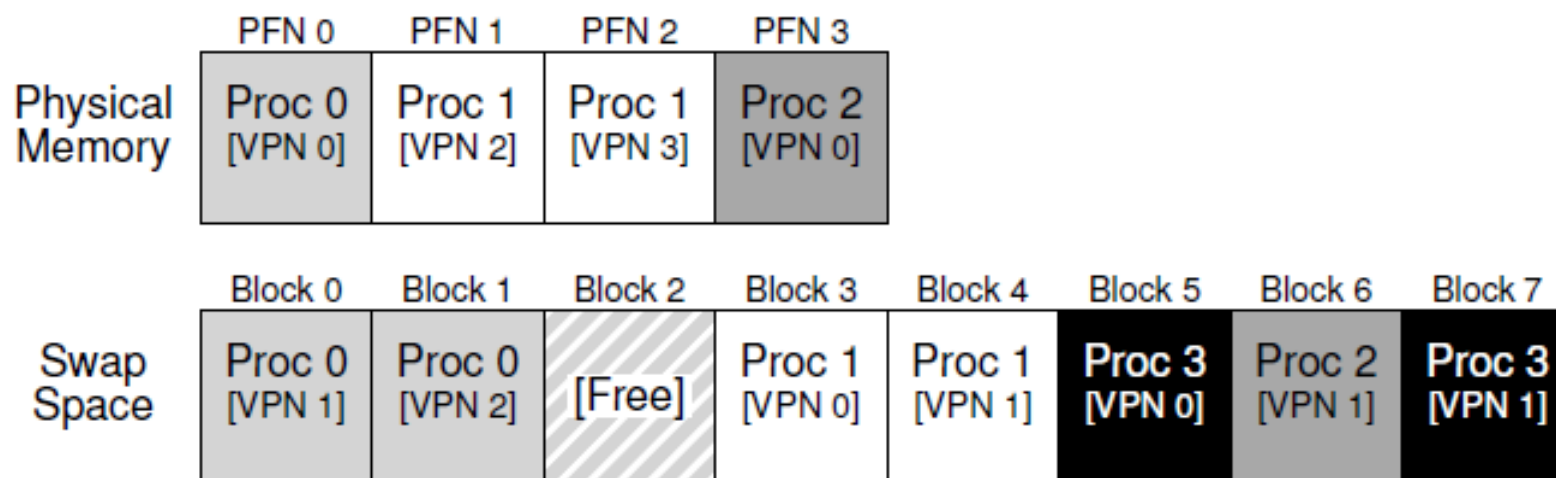
- A process is transferred temporarily from main memory to backing storage (disk) and another process is transferred from backing storage into main memory

Process Swapping



Page Swapping

- It is possible to execute a partially loaded process



- In this example, only parts of Processes 0, 1, and 2 are in physical memory and some pages are swapped out to disk
- Process 3 has all of its pages swapped out to disk (and therefore cannot be executed)

Virtual Memory using Paging

Frame #	Present bit
	1
	1
	1
	1
	0
....	
	0
	0

page table

- (main mem)
present bit-ny physical mem gshwyls

A **present bit** is associated with each page table entry (PTE)

 - 1 = in main memory (present)
 - 0 = not in main memory
- During address translation

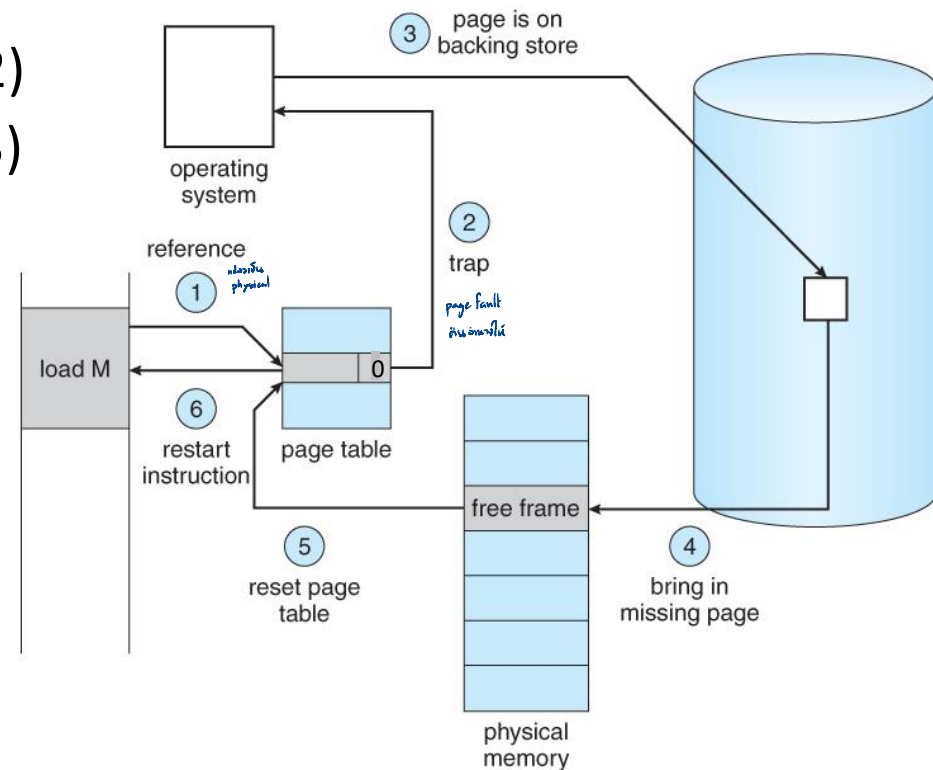
 - If present bit is 1, this is a **hit**
 - If present bit is 0, this is a **miss** and causes a **page fault**

page gshwyls main mem
notyyls Virtual Mem (disk)
 - Page fault trapped by OS and handled by **page fault handler**

→ in trap → gshwyls

Page Fault

- On a memory reference (1), if the present bit in the PTE for the logical page is 0, there is a page fault that is trapped by the OS (2)
- The OS finds the page needed (3)
 - The bits in the PTE normally used for the PFN can instead hold the disk address
- The OS brings the page into physical memory (4)
- The PTE is reset (5) to the PFN that now contains the page and the present bit set to 1
- The instruction is restarted (6)



Page Fault Handler

- Save the process context (registers, process state, etc.)
- Determine the location of the page on the disk
- Issue a read request from the disk to a free frame
- Put the process on a wait queue for this device until the read request is serviced
 - While the process is waiting, allocate the CPU to some other user
- Receive an interrupt from the disk I/O subsystem (I/O completed)
- Reset the page table to show the page is now in memory
- When the CPU is allocated to this process again
 - Save the context for the current user
 - Restore the context and page table registers for the process that caused the page fault, and then restart the interrupted instruction

Page Selection Policies

(determine when a page is swapped in)

- **Demand Paging** *via Page fault into swap-in*
 - Never swap a page in, unless the page will be needed
 - A page is swapped in only when a page fault occurs
- **Request Paging** *by programmer intentionally swap-in into*
 - Swap in a page if the user process requests it
 - However, user processes do not always know best
- **Pre-paging (or Pre-fetching)** *manually: swap-in later*
 - Swap in the next page before it is referenced
 - Uses concept of locality of reference (spatial)
 - However, there is a possibility of doing wasted work

Page Replacement Policies

(determine the best page to swap out)

- Suppose the physical memory becomes full (or nearly full)
- The OS needs to swap out one or more pages to make room for the new page(s)
 - Can use a *modify bit* to reduce overhead – only modified pages are written back to disk
- The process of choosing a page to swap out is known as the Page Replacement policy (PR) – some PR policies are:
 - First-In-First-Out PR (FIFO)
 - Least-Recently-Used PR (LRU) *แก้ไขล่าสุด*
 - Least-Frequently-Used PR (LFU) *แก้ไขบ่อย (มี counter กับ frequency)*
 - Optimal PR (OPT) – idealistic, but useful for comparison

แก้ไขบ่อย :: แก้ไขบ่อย Page select ไม่ดี in real world

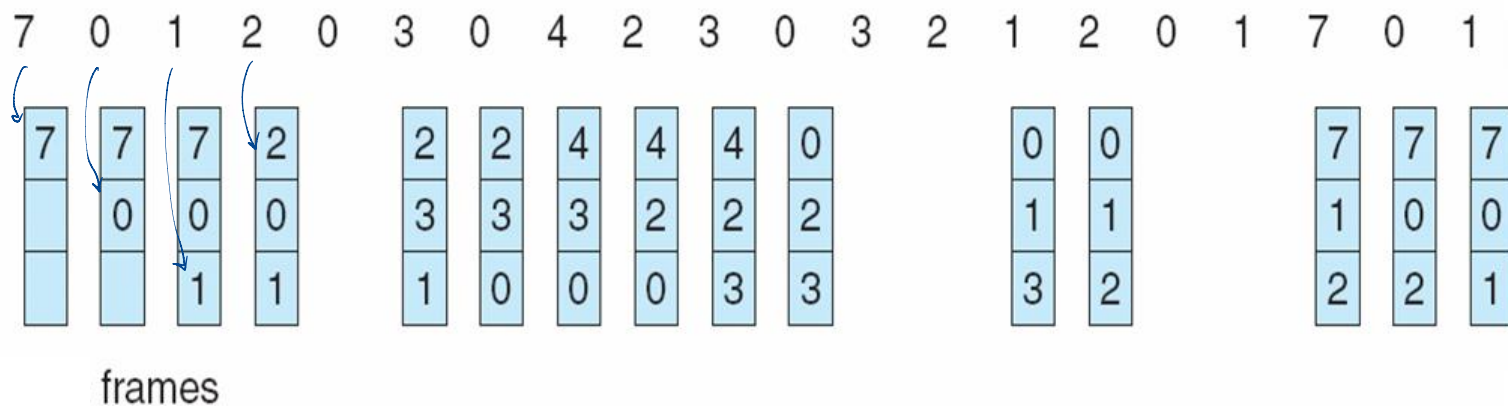
Example Reference Strings

- We will use two example strings of logical page references (reference strings) to evaluate the different PR policies
 - Example 1: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1
 - Example 2: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
- The numbers in these strings identify logical pages (of possibly different processes)
- We will compare the number of page faults assuming 3 (or 4) physical frames in physical memory
- The first references are always misses, as all physical frames are initially empty – such a miss is called a **cold-start miss** (or **compulsory miss**)
ตามแบบฉบับแล้ว

First-In-First-Out PR Policy

- The page number is added to a queue when the page is swapped into memory
- When we need to replace a page, the first page on the queue is swapped out

reference string

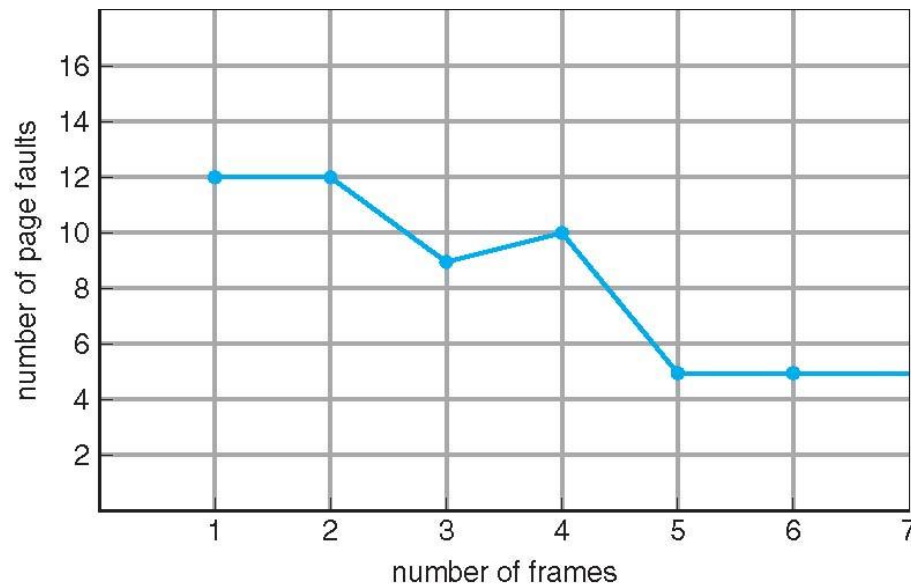


- Number of page faults = 15 and hit rate = $5/20 = 25\%$

אלגוריתם: FIFO

Belady's Anomaly

- This is an anomaly as, intuitively, increasing the number of frames should decrease the number of page faults



- In this case, as we increase from 3 to 4 frames, the number of page faults also increases

Belady's Anomaly

- Consider Example 2: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

Assuming 3 frames:
9 page faults

1	4	5
2	1	3
3	2	4

Assuming 4 frames:
10 page faults

1	5	4	
2	1	5	
3	2		
4	3		

Least-Recently-Used PR Policy

- LRU tries to avoid swapping out an important page by using history information
- The idea is that the more recently a page has been referenced, the more likely it will be referenced again soon
- LRU can be implemented either by associating time of last use with each page or by using a stack
 - The page number is placed on the top of stack when the page is swapped into memory
 - If a page is referenced, its number is moved to the top of stack
 - When we need to replace a page, the bottom page on the stack is swapped out

Least-Recently-Used PR Policy

- Example 1:**

FIFO would replace 0 by 3

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

page frames

7	7	7	2		2		4	4	4	0			1		1		1		
	0	0	0		0		0	0	3	3			3		0		0		
		1	1		3		3	2	2	2			2		2		7		

Number of page faults = 12 and hit rate = $8/20 = 40\%$

What about Example 2 – does LRU suffer from Belady's Anomaly?

Least-Recently-Used PR Policy

- Consider Example 2: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

Assuming 3 frames:
10 page faults

1	4	5	3
2	1		4
3	2		5

Assuming 4 frames:
8 page faults

Because of its stack property,
LRU does not suffer from
Belady's anomaly

1			5
2			
3	5	4	
4	3		

Least-Frequently-Used PR Policy

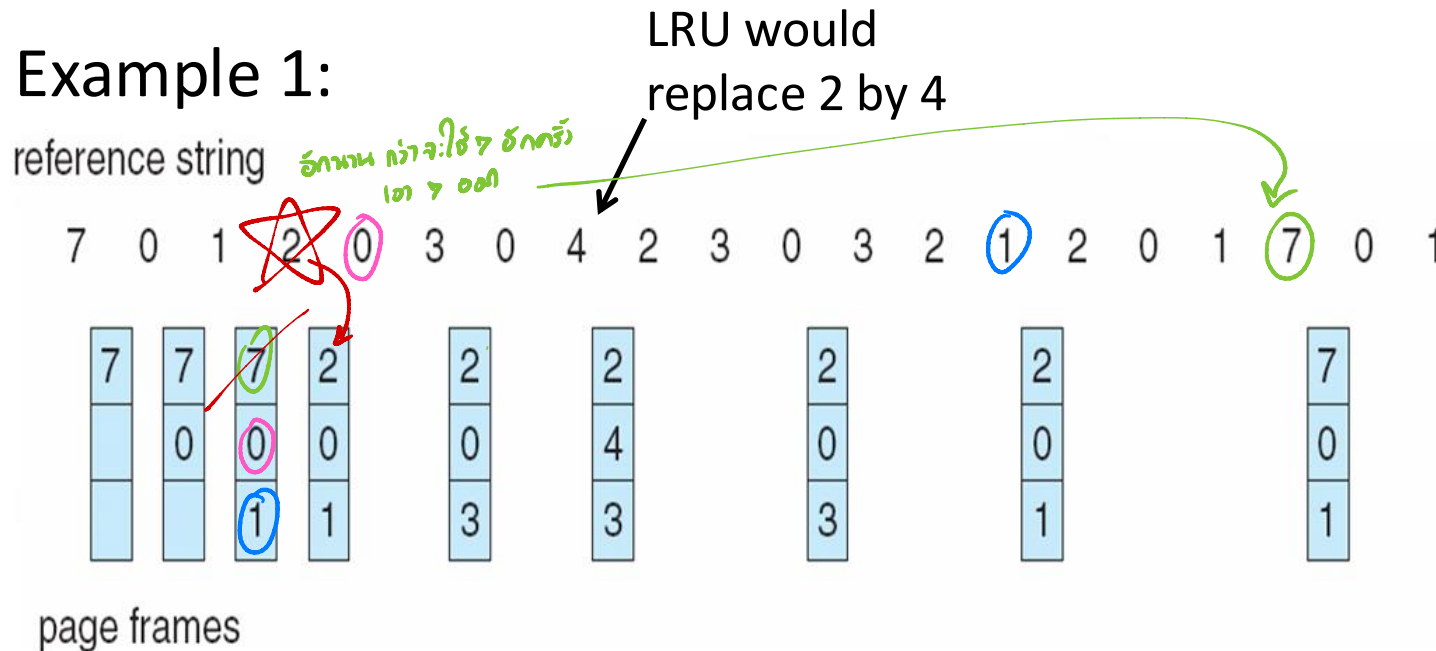
- Keep a count of the number of references that have been made to each page
- When we need to replace a page, the page with the smallest count is swapped out
- If all pages have the same count, use some other algorithm (e.g. FIFO) to handle this
- Example 1: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1
 - How many page faults (assuming 3 frames)?
- Example 2: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
 - How many page faults (assuming 3 or 4 frames)?

Optimal Replacement Policy

- The Optimal RP replaces the page that will be referenced furthest in the future
- This results in the fewest possible page faults and is therefore the best possible replacement policy
- Unfortunately, the future is not generally known, so it is not possible in practice to use the Optimal RP for any general purpose operating system
- However, to better understand how a particular PR policy such as FIFO, LRU or LFU behaves, it is useful to compare it against the Optimal RP

Optimal Replacement Policy

- Example 1:

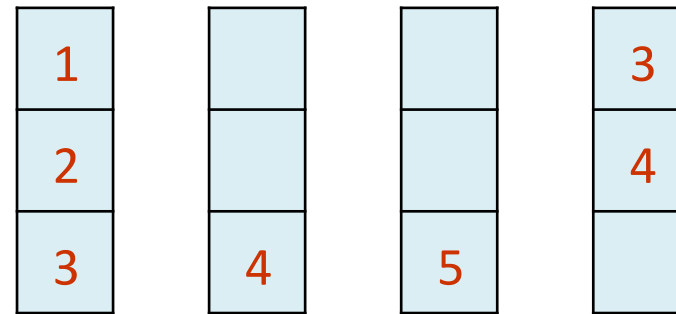


- Number of page faults = 9 and hit rate = $11/20 = 55\%$
- What about Example 2 – does OPT suffer from Belady's Anomaly?

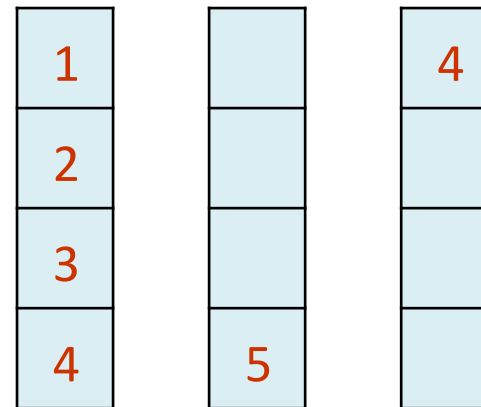
Optimal Replacement Policy

- Consider Example 2: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

Assuming 3 frames:
7 page faults



Assuming 4 frames:
6 page faults



Because it is optimal, OPT
does not suffer from Belady's
anomaly

Approximate to LRU

LRU Approximation Policies

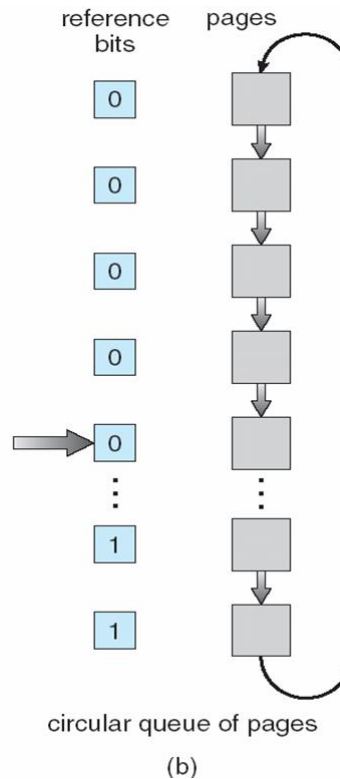
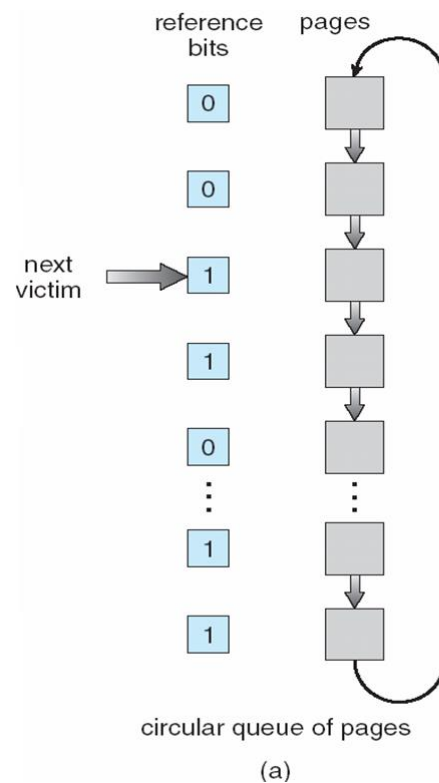
- LRU is effective, but can be slow, even when using a stack
 - Suppose the stack is implemented as a double linked list
 - When a page is referenced and its page number moved to the top of stack, 6 pointers need to be changed
- Some LRU Approximation Policies use a reference bit instead
 - Associate a reference bit with each page number, initially 0
 - When page is referenced, set bit to 1 – periodically clear to 0
 - When we need to replace a page, swap out any page with reference bit = 0 (if one exists)
 - This *approximates* LRU as the bit only indicates that the page has been referenced, not the order in which pages are referenced
 - The reference bit may be implemented in hardware for speed

Second-Chance RP

- Second-Chance uses a queue, similar to FIFO RP
 - The page number is added to a queue when the page is swapped into the memory, with reference bit set to 0
 - If a page is referenced, the reference bit is set to 1
 - When we need to replace a page
 - Examine the first page on the queue
 - If reference bit = 0, then replace the page
 - If reference bit = 1, then
 - Set reference bit to 0 and leave page in memory
 - Move page number to back of queue
 - Repeat
- Implement with **Circular Queue**

Circular Queue

- A circular queue is an efficient implementation of the Second-Chance policy



- The size of queue is equal to the number of frames
- The "hand" of the clock points to the front of queue
- To move a page number from the front to the back of queue (if reference bit = 1), the hand is simply advanced to the next page
- This avoids the expense of changing pointers

More... Replacement Policies

- Most-Frequently-Used PR (MFU)
- Not-Recently-Used PR (NRU)
- Far Page PR
- Random PR
- More...

ศุภณัฐ

Effective Access Time

- Suppose the CPU's registers can be accessed in about 0.5 ns, cache in 5 ns, main memory in 50 ns, but a disk access is about 50 ms
- The Effective Access Time (EAT) is the average time to access a page of memory
- Sometimes a page fault occurs, so the time will be determined by the disk access time (da) instead of the main memory access time (ma)
- Let pf be the page fault probability, or miss rate – note that $(1-pf)$ is the hit rate

main mem & Hard disk

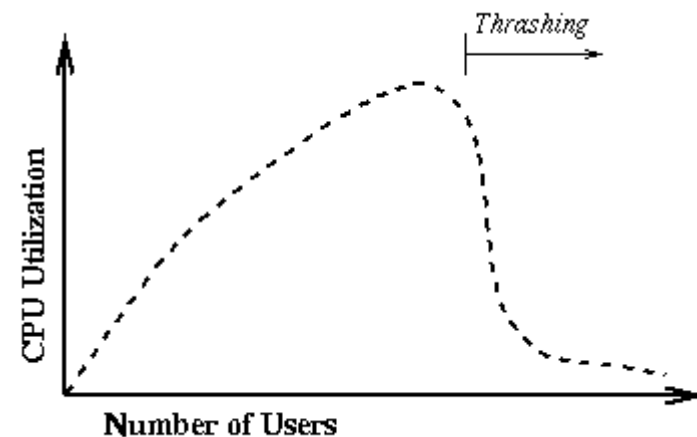
$$EAT = (1-pf)*ma + pf*da$$

- What is the pf value, if we need EAT to be not more than twice the main memory access time?

OS add process וזו: Page fault תדירות
OS swap in/out חוזר: .. וזו: Page fault תדירות

Thrashing Problem

- If a process does not have “enough” pages, the page fault (miss) rate is very high
 - Page reference causes page fault
 - Replace an existing page
 - But quickly need this page again
 - This leads to:
 - Low CPU utilization
 - OS thinking that it needs to increase the degree of multiprogramming – another process is added to the system
- **Thrashing** $\equiv$ a process is busy swapping pages in and out



Supplementary Section

Allocation of Frames

- **Global Replacement**
 - A replacement frame for a process is selected from the set of all frames in physical memory – one process can take a frame away from another
 - Unused frames are available to all processes
- **Local Replacement**
 - Each process is allocated its own set of frames
 - Equal Allocation (processes get the same number of frames)
 - Proportional Allocation (according to process size or priority)
 - A replacement frame for a process is selected from only its own set of allocated frames
 - Performance of a process does not depend on other processes

Working Set

- The **Working Set** WS_i of a process P_i is the number of pages referenced in the most recent time interval Δ
 - Informally, it is the number of pages a process is *actively* using
 - Δ is set by the OS to include the current locality of a process
 - Let $D = \sum_i WS_i$, for all currently executing processes P_i
 - If $D > m$, where m is the number of frames in physical memory, thrashing will occur
 - Policy – If $D > m$, suspend or swap out one of the processes
 - The Working Set may also be used in frame allocation
 - With Local Replacement, as the working set of a process grows and shrinks it can be allocated more or fewer frames accordingly

Considerations

- **TLB** – Ideally, the Working Set WS_i of each process P_i should fit in the Translation Lookaside Buffer, $WS_i \leq \text{TLB Size}$
 - **TLB Reach** is the amount of **memory** accessible from the TLB
 - $\text{TLB Reach} = (\text{TLB Size}) \times (\text{Page Size})$
- **I/O Overhead** – Number of I/O operations to load a process's working set into memory
- **Locality** – Important for the performance of TLB and some Page Replacement Algorithms
- **Page Size** – What is the effect of increasing page size on:
 - Fragmentation
 - Page table size
 - I/O overhead
 - Locality
 - Working set size
 - TLB size and performance
 - Number of page faults
 - etc.?